

Zigbee期末复习笔记

Zigbee期末复习笔记

期末考题型

第一章：传感器与感知技术

- 一、基础概念
- 二、传感技术与感知技术的区别

第二章：ZigBee无线传感网络

- 一、无线传感器技术（WSN）
- 二、无线传感网络系统
 - 1、组成
 - 2、工作流程
- 三、传感器节点
- 四、无线传感网络体系结构
- 五、IEEE 802.15.4无线传感器网络通信标准
- 六、ZigBee通信标准

第四章：CC2530基础知识

- 一、通用I/O端口
- 二、外部中断
- 三、定时器
- 四、USART
- 五、ADC
- 六、看门狗

第六章：Z-Stack协议栈基础

- 一、Z-Stack协议栈基本概念
 - 1、设备类型
 - 2、信道
 - 3、PANID
 - 4、地址
 - 5、任务
 - 6、事件&消息
- 二、Z-Stack协议栈工作原理（重点）
 - 一）系统初始化（zmain.c文件）
 - 二）任务初始化（在每个函数中）
 - 三）OSAL操作系统启动
 - 四）任务轮询和事件处理
 - 五）事件的产生与处理（important）
- 三、Z-Stack的通信
 - 一）数据的发送
 - 二）数据的发送方式
 - 三）Z-Stack协议栈串口通信

第七章：Z-Stack协议栈应用

- 一、NV操作函数
 - 1) NV操作的定义
 - 2) NV操作函数
 - 二、ZigBee网络设备地址
 - 三、分布式分配机制
-

期末考题型

其中绑定发送不考、协议栈基础的代码部分是重点

- 1、选择：10分
- 2、填空：10分
- 3、判断：10分
- 4、简答：20分
- 5、程序分析：50分（就是考察协议栈的东西）

第一章：传感器与感知技术

主要是一些概念、理解概念即可

一、基础概念

- **物联网**：通过各种传感设备、按照约定的协议，把任何物体与互联网相连接，进行信息交换和通信。广义：万物互联；狭义：物体之间相互连接的局域网
- **传感器**：能够感受规定的被测量并按照一定的规律转换成可用输出信号的器件或装置，通常由**敏感元件**和**转换元件**组成。
 - 敏感元件：用来感受测量量的传感器
 - 转换元件：把敏感元件感受到的探测量转换成适于传输和测量的电信号部分

二、传感技术与感知技术的区别

考察简答题

- **传感技术**侧重于信息的获取与转换，即将非电量转换为可用输出信号
- **感知技术**侧重于利用传感技术获取的信息进行识别和分析，以实现目标的准确识别。
- **传感技术**更多作为信息获取的基础手段，广泛应用于各种需要信息检测的场合。
- **感知技术**更多地与智能系统、自动化控制等领域相结合，用于实现复杂的目标识别和决策支持。

补充：感知技术的定义：感知技术是通过物理、化学或生物效应感受事务的状态、特征和方式的信息，按照一定的规律转换成可利用的信号，用以表征目标外部特征信息的一种信息获取技术。

第二章：ZigBee无线传感网络

对应课本的2、3章，ZigBee+IEEE标准

一、无线传感器技术（WSN）

- 定义：在检测范围内由大量廉价微型传感器节点组成，通过无线通信的方式形成的多跳的自组织网络系统。（**大规模、自组织、多跳**）
- 特点：
 - **节点数量多，网络密度高**
 - **分布式的拓扑结构**：没有固定的网络基础设施
 - **自组织特性**：在无人干预和其他任务网络基础设施支持，可以随时联网

二、无线传感网络系统

包含**传感器节点、汇聚节点和管理节点**

1、组成

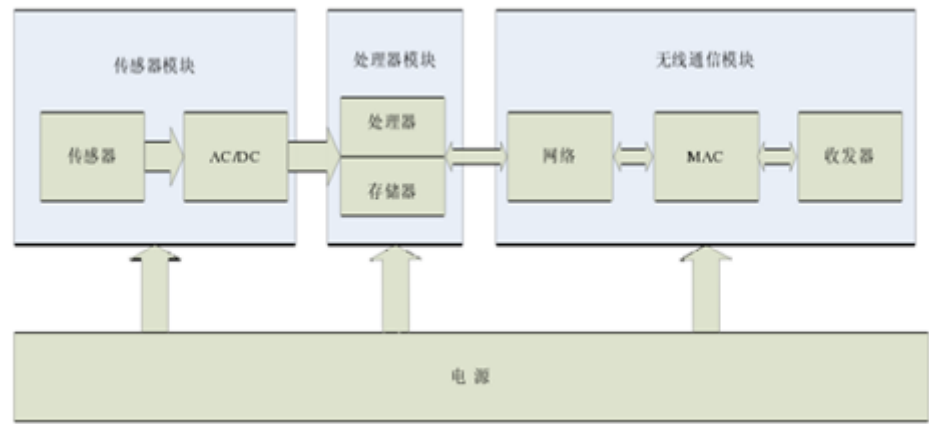
- **传感器节点**：无线传感器网络的基本单元，负责监测和采集环境数据。
- **汇聚节点**：将接收到的多个传感器节点的数据进行融合及压缩，通过因特网或卫星等通信方式传送到管理节点或用户数据处理中心
- **管理节点**：负责最终的数据处理和管理，提供用户界面或接口，供用户查询和分析数据

2、工作流程

传感器节点**采集数据**，通过其他传感器节点**逐跳地在网络中传输**，传输过程中数据可能被多个节点处理，经过多跳后**路由到汇聚节点**，最后**通过互联网或者卫星到达数据处理中心**。也可以沿着相反的方向，通过管理节点对传感器进行管理，发布监测任务以及收集监测数据。

三、传感器节点

包含**传感器模块、处理器模块、无线通信模块和电源模块**



- **传感器模块**：负责采集监测区域的信息，进行数据格式转换（将原始模拟信号转成数字信号）
- **处理器模块**：
 - 处理器：负责处理节点的控制
 - 存储器：负责节点的数据存储工作
- **无线通信模块**：专门负责节点之间的相互通信
- **电源模块**：为传感器节点提供能量，一般是采用微型电池供电

四、无线传感网络体系结构

这部分只要记**层内网络协议**即可

- **物理层**提供简单但健壮的信号调制和无线收发技术（数据调制发送接收）
- **数据链路层**负责数据成帧、帧检测、媒体访问和差错控制（数据检测）
- **网络层**主要负责路由生成与网络选择（多跳的无线路由协议）
- **传输层**负责数据流的传输控制，是保证通信服务质量的重要部分（数据流的传输控制）
- **应用层**包括一系列基于监测任务的应用层软件

五、IEEE 802.15.4无线传感器网络通信标准

通信协议是**短距离无线通信的IEEE标准**，是无线传感器网络通信协议中物理层与MAC层的一个具体实现。协议栈架构如下：

IEEE 802.15.4 标准结构	
高层	
网络层	
介质访问层	
868/915 MHz 物理层	2450 MHz 物理层

- 两种拓扑结构

- 星型网络拓扑结构：由中心设备PAN网络协调器集中控制，都与PAN进行通信



- 点对点网络拓扑：只要彼此都在对方的无线辐射范围之内，任何两个设备之间都可以直接通信

- 频段与数据传输率

1. 2.4GHz ISM频段：全世界范围内使用，数据传输率为250kb/s，信道数为16
2. 868MHz频段：全欧洲使用，数据传输率为20kb/s，信道数为1
3. 915MHz ISM频段：北美使用，数据传输率为40kb/s，信道数为10

- 设备类型

- 简化功能器件（RFD）：支持较少的通信功能和参数，通常用于低功耗、低成本的应用场景
- 全功能器件（FFD）：支持所有的通信功能和参数，可以作为协调器或路由器使用，以支持更复杂的网络拓扑和通信需求。

- 协议栈架构各层规范

- **物理层（PHY）**：负责在一条物理传输媒体上，实现数据链路实体之间透明地传输各种数据的比特流；提供以下服务：
 - **物理层数据服务**：从无线物理信息信道上收发信息
 - **物理层管理服务**：维护一个由物理层相关数据组成的数据库
- **MAC层规范**
 - **MAC层数据服务（MCPS）**：确保MAC协议数据单元在物理层数据服务中的正确收发
 - **MAC层管理服务（MLME）**：维护一个存储MAC子层协议状态相关信息的数据库

六、ZigBee通信标准

- ZigBee标准概述

- IEEE标准只处理**低级MAC层和物理层协议**，ZigBee联盟对**网络层协议和API进行了标准化**
- ZigBee技术特点

- **功耗低**：有休眠模式（不需要接收数据时处于休眠，需要接收数据时由协调器唤醒）
- **成本低**
- **可靠性高**：采取碰撞避免机制，MAC层采用完全确认的数据传输机制
- **容量大**：1个ZigBee网络最多可以容纳254个从设备和1个主设备；一个区域内最多可以同时存在100个ZigBee网络
- **时延小**
- **兼容性好**

• ZigBee协议框架

主要是根据这个图，知道什么协议对应什么层

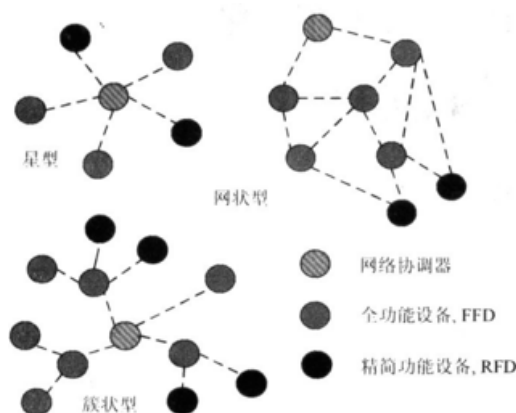


• ZigBee各层规范

1、网络层规范

网络层主要实现节点加入、离开、路由查找和传送数据等功能。

- 路由算法：**树路由**和**网状网路由**
- 拓扑结构：星型、树形、网格



- 拓扑结构一般包括三种设备：
 - **协调器（蜂后）**：FFD、ZigBee网络的启动或建立网络的设备
 - **路由器（雄峰）**：作为普通设备加入网络，实现多跳路由
 - **末端节点（工蜂）**：RFD，没有指定的责任
- **网络层规范概述**
 - ZigBee协议栈的核心部分在网络层。负责拓扑结构的建立和维护、命名和绑定服务。
 - **网络层数据实体（NLDE）**：提供相关的SAP（服务存取点）的数据传输服务；
 - 产生网络层协议数据单元（NPDU）

- 提供基于拓扑结构的路由策略。
- **网络层管理实体 (NIME)**：提供经由SAP的管理服务；
 - 配置新设备
 - 建立、加入和离开网络
 - 寻址
 - 邻居发现、路由发现
 - 接受控制

2、应用层规范

ZigBee应用层包括**APS子层**、**ZDO（包括ZDO管理层）**以及**用户自定义的应用对象**；三个组成部分：**应用支持子层APS**、**应用框架AF**、**ZigBee设备对象ZDO**

- 应用支持子层APS：
 - 协议数据单元APDU的处理
 - 网络层与应用层接口
 - APS数据实体 (APSDE)
 - APS管理实体 (APSME)
- 应用框架AF：提供了两种标准服务类型
 - 键值对 (KVP) 服务类型：用来传输规范所定义的特殊数据
 - 报文 (MSG) 服务类型
- ZigBee设备对象ZDO

主要功能集中在**网络管理和维护上**。应用层的端点可以通过ZDO提供的功能来**获取网络或者是其他节点的信息**

第四章：CC2530基础知识

一、通用I/O端口

【注意】每个指令周期是一个时钟，而标准的8051每个指令周期是12个时钟

- I/O端口引脚（21个）
 - 其中：P0（8位），P1（8位），P2（5位）
 - P1_0, P1_1(20mA)，其余的为(4mA)
 - I/O引脚的重要特性：
 - 可以**设置特殊功能寄存器（SFR）**来将这些引脚设为**普通I/O口**还是作为**外设I/O口**使用
 - 在输入时具有上拉和下拉的能力
 - 具有外部中断的能力（21个I/O引脚可以作为外部中断源输入口，外部中断可以将CC2530从睡眠模式中唤醒）
- 通用I/O端口相关寄存器
 - **PxSEL**：设置每个端口引脚为通用I/O或者外部设备的I/O
 - 0：通用I/O
 - 1：外设功能

POSEL (0xF3) - 端口 0 功能选择

位	名称	复位	R/W	描述
7: 0	SELP0_[7:0]	0x00	R/W	P0.7到 P0.0功能选择 0: 通用I/O 1: 外设功能

- **PxDIR**: 设置每个端口引脚为输入或输出

- 0: 输入
- 1: 输出

PDIR (0xFD) - 端口 0 方向

位	名称	复位	R/W	描述
7:0	DIRP0_[7:0]	0x00	R/W	P0.7到P0.0的I/O方向 0: 输入 1: 输出

- **PxINP**: 每个端口引脚为输入时, 设置为上拉、下拉或三态模式

- 0: 上拉/下拉 (由P2INP [7:5] 设置)

P2INP (0xF7) - 端口 2 输入模式

位	名称	复位	R/W	描述
7	PDUP2	0	R/W	端口2上拉/下拉选择。对所有端口2引脚设置为上拉/下拉输入。 0: 上拉 1: 下拉
6	PDUP1	0	R/W	端口1上拉/下拉选择。对所有端口1引脚设置为上拉/下拉输入。 0: 上拉 1: 下拉
5	PDUP0	0	R/W	端口0上拉/下拉选择。对所有端口0引脚设置为上拉/下拉输入。 0: 上拉 1: 下拉
4:0	MDP2_[4:0]	0 0000	R/W	P2.4到P2.0的I/O输入模式 0: 上拉/下拉 1: 三态

- 1: 三态

P0INP (0x8F) - 端口 0 输入模式

位	名称	复位	R/W	描述
7:0	MDP0_[7:0]	0x00	R/W	P0.7到P0.0的I/O输入模式 0: 上拉/下拉(见P2INP (0xF7)-端口2输入模式) 1: 三态

【总结】

PxSEL (设置通用/外设)、PxDIR (设置输入/输出)、PxINP (设置上下拉/三态)

二、外部中断

- 中断屏蔽:
 - 一共有18个中断源
 - 每个中断请求可以通过设置**中断使能寄存器**IEN0, IEN1或者IEN2的中断使能位 使能或禁止 (**0禁用1使能**)
- 中断使能步骤:
 - (1) 使用IEN0中**IEN0.EA位为1**, 开中断
 - (2) 设置寄存器IEN0、IEN1和IEN2中**相应中断使能位为1**
 - (3) 如果需要, 则设置P0、P1、P2**各引脚对应的各中断使能位为1**
 - (4) 最后在寄存器PICTL中**设置中断是上升沿还是下降沿触发**



【总结】 (?)

记住下面这些寄存器：

IEN0 (EA / URX1IE / URX0IE / ADCIE) 、 IEN1 (P0IE / T1IE) 、 IEN2 (WDTIE / UTX1IE / UTX0IE / P1IE / P2IE) 、 PxIEN、中断标志

三、定时器

USART、定时器和ADC这样的片内外设同样也需要I/O口实现其功能。

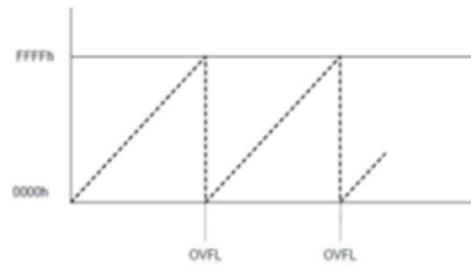
I/O引脚被选择实现片内外设I/O功能，需要**设置对应的PxSEL位为1**

- 定时器简介：CC2530一共有四个定时器 T1、T2、T3、T4
 - T1：**16位定时/计数器**，支持输入采样、输出比较和PWM功能。
 - T1有五个独立的输入采样/输出比较通道，每一个通道对应一个I/O口。
 - T2：为**MAC定时器**
 - T3、T4：为8位定时/计数器，支持输出比较和PWM功能。
- 定时器1寄存器：
 - T1CNTH：定时器1计数高位寄存器
 - T1CNTL：定时器1计数低位寄存器
 - T1CTL：定时器1控制寄存器（3，2位分频、1，0位模式）
 - T1STAT：定时器1状态寄存器
- 定时器1的操作：

一般来说，定时器1有三种操作模式：**自由运行模式、模模式、正计数/倒数模式**

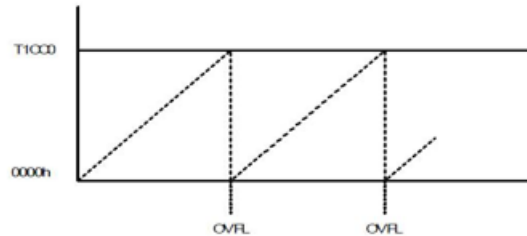
 - 自由运行模式 (MODE[1:0]=01)

计数器从0x0000开始，每个活动时钟边沿增加1。当计数器**达到0xFFFF（溢出）**时，计数器载入0x0000，继续递增他的值。（P76，图5.8）



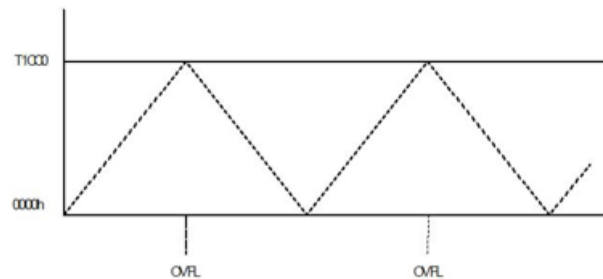
- 模模式 (MODE[1:0]=10)

计数器从0x0000开始，每个活动时钟边沿增加1.当计数器达到寄存器T1CC0（溢出）时，寄存器T1CC0H: T1CC0L保持最终计数值，计数器将复位到0x0000，并继续递增。
(P77, 图5.9)



- 正计数/倒数模式 (MODE[1:0]=11)

计数器反复从0x0000开始，正计数直到到达T1CC0H:T1CC0L保存的值。然后计数器将倒数直到0x0000



四、USART

- USART寄存器

对于每个USART，有5个如下的寄存器（x是USART的编号，为0或1）

- UxCSR: USARTx控制和状态;
- UxUCR: USARTxUART控制;
- UxGCR: USARTx通用控制
- UxDBUF: USART x 接收/发送数据缓冲
- UxBAUD: USART x 波特率控制

在运行UART模式时，内部的波特率发生器设置UART波特率

- UART模式

-

五、ADC

A/D转换器就是模拟/数字转换器，是将输入的模拟信号转换成为数字信号。

- ADC寄存器
 - ADCCON1：ADC控制1
 - ADCCON3：
 - ADCL（高6位计数）：ADC数据低位
 - ADCH：ADC数据高位
- ADC的输入
 - 单端输入（AIN0 ~ AIN7）：通道0 ~ 通道7
 - 差分输入对（AIN0 ~ AIN1、AIN2 ~ AIN3、AIN4 ~ AIN5和AIN6 ~ AIN7）：通道8 ~ 11
 - GND：通道12
 - 温度传感器：通道14
 - AVDD5/3：通道15
 - **ADC使用哪个通道作为输入由寄存器ADDCON2（序列转换）或ADDCON3（单个转换）决定**

六、看门狗

当单片机程序可能进入死循环的情况下，看门狗定时器（WDT）用作一个恢复的方法。当软件在选定时间间隔内不能清除WDT时，WDT必须复位系统。

- 看门狗模式

这个模式下：WDT运行在一个频率32.768kHz（当使用32kHz XOSC）的看门狗定时器时钟上。**超时期限等于1.9ms，15.625ms，0.25s和1s**，分别对应64，512，8192和32768的计数值设置。

计数器达到选定定时器的间隔值，看门狗定时器就为系统产生一个复位信号。看门狗清除的序列包括在一个看门狗时钟周期内，**写入0xA到WDCTL.CLR[3:0]，然后写入0x5到同一个寄存器位。**

在看门狗模式下，WDT**不会产生一个中断请求。**

第六章：Z-Stack协议栈基础

一、Z-Stack协议栈基本概念

Z-Stack协议栈是一个基于任务轮询方式的操作系统：OSAL

工作原理：任务轮询 >>> 事件处理函数 >>> 任务同步

1、设备类型

- **协调器**：协调器的角色主要**涉及网络的启动和配置**。一旦这些都完成后，协调器的工作就和**一个路由器相同**
- **路由器**：允许其他设备加入网络，多跳路由协助由电池供电的子终端设备的通讯。
- **终端设备**：通常用于**执行特定的任务或功能**，如传感器数据采集、控制设备操作等。**终端设备没有维持网络结构的职责**

2、信道

1. 2.4GHz ISM频段：全世界范围内使用，数据传输率为250kb/s，信道数为16
2. 868MHz频段：全欧洲使用，数据传输率为20kb/s，信道数为1
3. 915MHz ISM频段：北美使用，数据传输率为40kb/s，信道数为10

3、PANID

一个16位的ID值来标识唯一——一个ZigBee网络，主要是用于区分网络，使得同一地区可以同时存在多个ZigBee网络。

4、地址

- IEEE地址：**64位地址使全球唯一的地址**，由制作商设置
- 网络地址：**16位地址，当设备加入网络后分配**

5、任务

协议栈中的**每一层都设计了一个事件处理函数**，用来处理与这一层操作相关的各种事件。将这些**事件处理函数看成是与协议栈每一层相对应的任务**。

6、事件&消息

- 事件是**驱动任务区执行某些操作的条件**，当系统产生了一个事件，将这个触发传递给相应的任务后，任务才能执行一个相应的操作。OSAL通过一个**16位宽度的数组**来管理事件，意味着OSAL最多可以支持16个事件，其中最高位（0x08000，SYS_EVENT_MSG）系统保留，用户可以使用的事件有15个。
- 消息则是事件编号和其对应事件内容的集合，发生事件并将其打包到消息队列，供事件处理函数进行处理：

事件发生后 >>> 打包为消息 >>> 存放到消息队列 >>> 事件处理函数取出消息并进行操作

二、Z-Stack协议栈工作原理（重点）

Z-Stack协议栈的工作原理主要**基于任务轮询方式**，其任务调度和资源分配由操作系统抽象层（OSAL）管理。

一）系统初始化（zmain.c文件）

Z-Stack由main（）函数开始执行，main（）函数一共做了两件事：一件是**系统初始化**，另一件是**开始执行函数osal_start_system()**，进入**轮转查询式操作系统**。需要完成以下初始化工作：

- 初始化系统时钟，检测芯片工作电压
- 初始化栈堆，各个硬件模块（如CC2530硬件模块）
- 初始化FLASH存储，形成芯片MAC地址
- 初始化非易失变量，MAC层协议和应用层协议
- 初始化操作系统等

```

int main(void)
{
    uint16 i,j;
    // Turn off interrupts
    osal_int_disable( INTS_ALL );

    // Initialization for board related stuff such as LEDs
    HAL_BOARD_INIT();

    // Make sure supply voltage is high enough to run
    zmain_vdd_check();

    // Initialize board I/O
    InitBoard( OB_COLD );
    for(i=0;i<1000;i++)
        for(j=0;j<6000;j++);

    // Initialize HAL drivers
    HalDriverInit();

    // Initialize NV System
    osal_nv_init( NULL );

    //Set channel,panid
    ParaProcess();

    // Initialize the MAC
    ZMacInit();

    // Determine the extended address
    zmain_ext_addr();

#ifdef ZCL_KEY_ESTABLISH
    // Initialize the Certicom certificate information.
    zmain_cert_init();
#endif

    // Initialize basic NV items
    zgInit();

```

二) 任务初始化 (在每个函数中)

任务初始化为每一层分配一个任务ID，分配任务ID时要和每一层的事件处理函数一一对应。

在主函数中调用文件osal.c中的函数 `osal_init_system()`

调用文件OSAL_SampleApp.c中的函数 `osalInitTasks()` 进行任务初始化

```

void osalInitTasks( void )
{
    uint8 taskID = 0; //这就是分配一个任务ID

    //这两句为一个长度为taskCnt(任务个数)的uint16数组tasksEvents分配了内存空间并将其值初始化为零。
    tasksEvents = (uint16 *)osal_mem_alloc( sizeof( uint16 ) * tasksCnt);
    osal_memset( tasksEvents, 0, (sizeof( uint16 ) * tasksCnt));

```

```

//给后面的每个taskID都递增加1，在每一层初始化过程中记录分配给它的taskID值
macTaskInit( taskID++ );
nwk_init( taskID++ );
Hal_Init( taskID++ );

#if defined( MT_TASK )
    MT_TaskInit( taskID++ );
#endif
    APS_Init( taskID++ );
#if defined ( ZIGBEE_FRAGMENTATION )
    APSF_Init( taskID++ );
#endif
    ZDApp_Init( taskID++ );
#if defined ( ZIGBEE_FREQ_AGILITY ) || defined ( ZIGBEE_PANID_CONFLICT )
    ZDNwkMgr_Init( taskID++ );
#endif
    SampleApp_Init( taskID ); //这就是调用SampleApp.c中的函数进行初始化

```

在文件OSAL_SampleApp.c中，有一个**常量数组**存放着**每一层事件处理函数的地址**，这样每一层的事件处理函数就可以通过每一层的TaskID来访问了。

```

//创建一个数组taskArr[]用来存放taskid
const pTaskEventHandlerFn tasksArr[] = {
    macEventLoop,
    nwk_event_loop,
    Hal_ProcessEvent,
#if defined( MT_TASK )
    MT_ProcessEvent,
#endif
    APS_event_loop,
#if defined ( ZIGBEE_FRAGMENTATION )
    APSF_ProcessEvent,
#endif
    ZDApp_event_loop,
#if defined ( ZIGBEE_FREQ_AGILITY ) || defined ( ZIGBEE_PANID_CONFLICT )
    ZDNwkMgr_event_loop,
#endif
    SampleApp_ProcessEvent //这段是自己写的代码所在的任务，负责具体的业务逻辑
};

```

三) OSAL操作系统启动

在main函数里，在最后调用了osal_start_system()函数，作为操作系统工作的启动

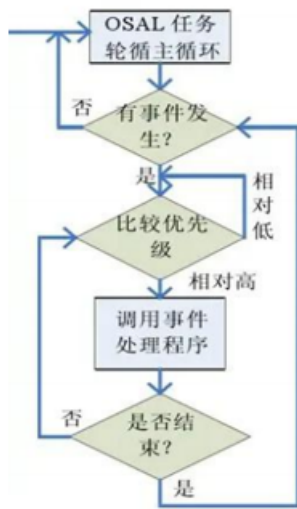
```

void osal_start_system(void)
{
}

```

作用：提供一个无限循环，可以不断地执行osal_run_system()函数

osal_run_system() 的工作流程



四) 任务轮询和事件处理

运行 `osal_run_system()` 程序

```

void osal_run_system( void )
{
    uint8 idx = 0;
    osalTimeUpdate();
    Hal_ProcessPoll();
    do {
        if (tasksEvents[idx]) // Task is highest priority that is ready.
        {
            break;
        }
    } while (++idx < tasksCnt);
    if (idx < tasksCnt)
    {
        uint16 events;
        halIntState_t intState;    //typedef unsigned char
        halIntState_t;

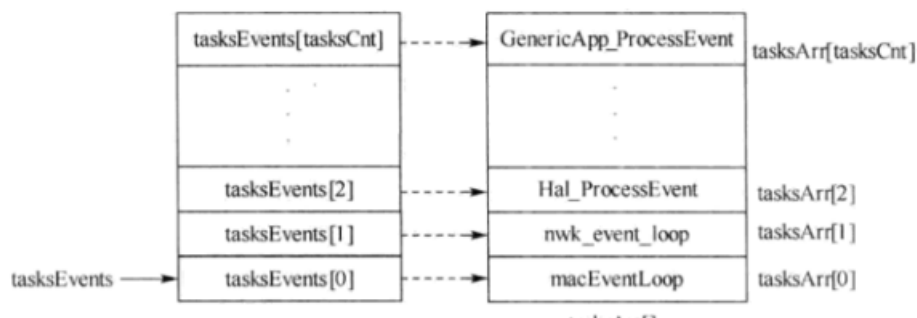
        HAL_ENTER_CRITICAL_SECTION(intState);
        events = tasksEvents[idx]; //保存事件标志。
        tasksEvents[idx] = 0;      // Clear the Events for this task.
        HAL_EXIT_CRITICAL_SECTION(intState);

        activeTaskID = idx;
        events = (tasksArr[idx])( idx, events ); //调用相应的事件处
        理函数。
        activeTaskID = TASK_NO_TASK; //define TASK_NO_TASK
        0xFF
        HAL_ENTER_CRITICAL_SECTION(intState);
        tasksEvents[idx] |= events; // Add back unprocessed events
        to the current task.
        HAL_EXIT_CRITICAL_SECTION(intState);
    }
}
  
```

【代码分析】：

- idx: 数组编号
- **tasksEvents[]**: 任务事件数组，储存着事件
- tasksCnt: 保存着任务总个数
- **tasksArr[]**: 任务队列指针数组，指向事件处理函数

当任务事件非空时，记录对应的idx，调用对应的层事件处理函数，处理完成返回未处理的函数。



事件表与任务表之间的关系

五) 事件的产生与处理 (important)

Z-Stack协议栈的核心是事件的产生和事件的处理。事件的产生可以通过多种方式，如自定义事件、立即事件（通过osal_set_event函数设置）和延时事件（通过osal_start_timeEx函数设置定时器，在定时器超时后产生事件）。

- 事件初始化: `void SampleApp_Init(uint8 task_id);`
- 事件处理函数: `uint16 SampleApp_ProcessEvent(uint8 task_id,uint16 events)`

```
if ( events & SYS_EVENT_MSG )           //处理系统事件
case KEY_CHANGE:                         //处理接收消息的事件
```

- `SYS_EVENT_MSG`：系统消息事件处理，事件发生后 >>> 被打包为消息 >>> 存放到消息队列
- `KEY_CHANGE`：按键改变事件
- `ZDO_STATE_CHANGE`：组网状态改变事件
- `AF_INCOMING_MSG_CMD`：接收消息事件

三、Z-Stack的通信

一) 数据的发送

调用函数 `AF_DataRequest()` 即可完成数据的发送

```
afStatus_t AF_DataRequest( afAddrType_t *dstAddr, endPointDesc_t *srcEP,uint16
cID,uint16 len, uint8 *buf, uint8 *transID, uint8 options, uint8 radius)
```

- `*dstAddr`：发送目的地址、端点地址以及传送模式
- `*srcEP`：源端点
- `cID`：簇ID
- `len`：数据长度
- `*buf`：数据

二) 数据的发送方式

- **单播**

单播需要直到目标设备的短地址，需要将dstAddr >>> addrMode设为 `Addr16Bit`，dstAddr->addr->shortAddr设置为目标设备的短地址即可。代码如下：

```
SampleApp_Flash_DstAddr.addrMode = (afAddrMode_t)Addr16Bit;
SampleApp_Flash_DstAddr.endPoint = SAMPLEAPP_ENDPOINT;
sampleApp_Flash_DstAddr.addr.shortAddr = 0x0000; //协调器的地址为0
```

根据上面代码的配置，使用 AF_DataRequest() 函数来进行点对点发送。

- **组播**

如果设备想传输数据到某一组设备，那么只需将dstAddr->addrMode设为 AddrGroup，dstAddr->addr->shortAddr设置为相应的组ID即可。代码如下：

```
SampleApp_Flash_DstAddr.addrMode = (afAddrMode_t)afAddrGroup;
SampleApp_Flash_DstAddr.endPoint = SAMPLEAPP_ENDPOINT;
SampleApp_Flash_DstAddr.addr.shortAddr = SAMPLEAPP_FLASH_GROUP;
```

根据上面代码的配置，使用AF_DataRequest()函数进行组播发送。

- **广播**

广播发送可以分为三种，将dstAddr->addrMode设为AddrBroadcast,dstAddr->addr->shortAddr设置为相应的广播类型即可。

- NWK_BROADCAST_SHORTADDR_DEVALL(0xFFFF)--数据包将被传送到网络上的**所有设备**，包括睡眠中的设备。对于睡眠中的设备，数据包将被保留在其父亲节点直到查询到它，或者消息超时。
- NWK_BROADCAST_SHORTADDR_DEVRXON(0xFFFD)--数据包将被传送到网络上的所有接收机的设备(RXONWHENIDLE)，也就是说，**发送到了除了睡眠中的所有设备**。
- NWK_BROADCAST_SHORTADDR_DEVZCZR(0xFFFC)--数据包发送给**所有的路由器，包括协调器**。

三) Z-Stack协议栈串口通信

- 串口初始化配置

```
halUARTCfg_t uartConfig;
uartConfig.configured = TRUE; //使能串口
uartConfig.baudRate = HAL_UART_BR_115200; //波特率 115200
uartConfig.flowControl = FALSE; //关闭流控
uartConfig.callBackFunc = ChatApp_rxCB; //回调函数
HALUARTOpen(HAL_UART_PORT_0, &uartConfig); //打开串口
```

- 串口使用

```
uint8 HALUARTOpen(uint8 port, halUARTCfg_t *config);
uint16 HALUARTRead(uint8 port, uint8 *buf, uint16 len);
uint16 HALUARTWrite(uint8 port, uint8 *buf, uint16 len);
```


第七章：Z-Stack协议栈应用

一、NV操作函数

1) NV操作的定义

Z-Stack协议栈中的NV操作主要涉及**非易失性存储器的读写和管理**，这对保存网络的配置参数以及用户数据至关重要。

在Z-Stack协议栈中，NV存储器**主要用于保存网络的配置参数**，如网络地址等，以便在**设备掉电重启后仍能读取这些参数，并自动加入到原来的网络中**。这样，设备的网络地址等关键信息在重启后不会丢失，从而保证了网络的稳定性和连续性。

2) NV操作函数

- `osal_nv_init`：该函数在系统初始化时被调用，用于初始化NV存储器
- `osal_nv_item_init`：该函数用于初始化NV存储器中的一个条目 (item)
- `osal_nv_read`：该函数用于读取NV存储器中的一个条目。函数原型为uint8
- `osal_nv_read(uint16 id,uint16 offset,uint16 len,void *buf)`：其中id是NV条目的ID，offset是读取的起始偏移量，len是要读取的数据长度，buf是指向存储读取数据的缓冲区的指针。
- `osal_nv_write`：该函数用于向NV存储器中写入数据。如果指定的NV条目不存在，，则会创建一个新的条目。函数原型为 `uint8 osal_nv_write(uint16 id,uint16 offset,uint16 len,void *buf)`，参数含义与osal_nv_read函数相同。

二、ZigBee网络设备地址

在ZigBee网络中，主要有三种类型的设备，设备类型的选择是在编译时根据不同的编译选项来确定的。

- 协调器：上电后，协调器会自动选择一个信道，然后选择网络号，建立网络。主要是在**网络建立、网络配置方面起作用**，一旦建立网络，协调器就与路由器的功能是一样的。
- 路由器：负责三个功能
 - 允许节点加入网络
 - 进行数据的路由
 - 辅助其孩子节点通信
- 终端节点：只需要加入已建立的网络即可，终端节点不具有网络维护功能。

三、分布式分配机制

这部分会考察一道计算题（有可能是大题）

记住公式，能够做题即可！

ZigBee无线网络中，**协调器**(Coordinator)在建立网络以后**使用0x0000**作为自己的网络地址(即协调器的默认网络地址是0x0000)。在**路由器(Router)**和**终端(Enddevice)**加入网络以后，父设备**会自动给它分配16位**的网络地址。

网络地址是16位的，因此最多可以分配给65536个节点，地址的分配取决于整个网络的架构，整个网络的架构由以下3个值决定：

网络的最大深度(L_m);

每个父节点拥有的孩子节点最大数目(C_m);

每个父节点拥有的孩子节点中路由器的最大数目(R_m)。

可以根据下面的公式来计算某父节点的路由器子设备之间的地址间隔 $C_{skip}(d)$

$$C_{skip}(d) = \begin{cases} 1 + C_m * (L_m - d - 1); & \text{当 } R_m = 1 \\ \frac{1 + C_m - R_m - C_m * R_m^{L_m - d - 1}}{1 - R_m} & \text{其他} \end{cases}$$

$$A_n = A_{parent} + C_{skip}(d) * R_m + n$$

终端节点的网络地址计算公式如下：

$$A_n = A_{parent} + C_{skip}(d) * R_m + n$$

以上公式是用来计算 A_{parent} 这个父亲设备分配的第 n 个终端设备的地址 A_n 。